

Flight Delay Prediction Using Open Transportation Data

Alina Iakubova

Computer Science and Engineering

Digital Transformation Management

University of Bologna

Email: alina.iakubova@studio.unibo.it

Abstract—This study develops a machine learning approach for predicting flight departure delays using historical airline, airport, weather, and operational data. The problem is formulated as a binary classification task, where a flight is classified as delayed if its departure delay is 15 minutes or more. The data preparation stage included missing value analysis, duplicate removal, type conversion, categorical encoding, feature scaling, outlier inspection, and feature engineering. Extreme but plausible observations, such as severe weather conditions, high traffic levels, or unusual airport activity, were retained because they may reflect real operational situations related to delays.

Several models were evaluated, including KNN, KNN with PCA, RIPPER, Gaussian Naive Bayes, Decision Tree, Random Forest, and LightGBM. Distance-based and probabilistic models showed limited recall, struggling to capture the complex interactions between features. Tree-based models performed substantially better, with the feature-engineered LightGBM achieving the best result: F1-score of 0.43 and ROC-AUC of 0.73 on the test set. Feature importance and SHAP analysis indicated that departure hour, precipitation, and departure airport were among the most influential predictors. Overall, the results show that machine learning can identify meaningful delay-related patterns, but prediction accuracy remains moderate without richer real-time operational data.

1. Introduction

Flight delay prediction is an important problem for the aviation industry because delays affect airlines, airports, passengers, and the overall efficiency of air transportation. Even a single delayed departure can create additional costs, disrupt passenger connections, reduce aircraft and crew availability, and contribute to congestion across the network. For passengers, delays often mean missed connections, longer waiting times, and lower service satisfaction. For airlines and airports, early identification of flights with a high delay risk can support better resource planning, operational decisions, and disruption management.

This machine learning project is dedicated to predicting whether a flight departure will be delayed by 15 minutes or more. The task is treated as a binary classification problem, where the target variable indicates whether a flight is delayed

or on time. Dataset is based on Bureau of Transportation statistics and provides information about USA flights in 2019. It includes airline, airport, weather, and operational data. The aim is also to explore which factors are associated with departure delays and to compare the performance of different machine learning models.

The implementation is organized into several parts:

- 1) **Data Import.** This stage loads the dataset from Kaggle and prepares it for further analysis. The project uses one CSV file, *full_data_flightdelay.csv*, which already contains both the target variable and the explanatory features. The target variable is `DEP_DEL15`, indicating whether a flight departure was delayed by 15 minutes or more. The dataset also includes flight, airport, airline, weather, and operational features, such as `DEPARTING_AIRPORT`, `CARRIER_NAME`.
- 2) **Data Understanding.** This section explores the structure of the data, target distribution, categorical feature distributions, and relationships between delays and variables such as departure hour, month, airport, carrier, and weather conditions. This stage helps identify initial patterns and gives context for later modeling decisions.
- 3) **Data Quality Analysis.** This section checks whether the dataset is reliable enough for modeling and identifies which types of transformations are needed before training machine learning models. This stage includes an analysis of missing values, duplicates, data types, negative values, suspicious zeros, and logical consistency across variables. For example, weather variables are checked for plausibility: snowfall values are compared with months and airport locations to confirm that snow appears in reasonable seasonal and geographic contexts. The main purpose of this stage is to make sure that the data can be trusted, and to detect potential data quality issues.
- 4) **Data Preparation.** This stage includes outlier detection, cardinality analysis and correlation analysis. Outliers are not automatically removed, because extreme values may represent meaningful real-world conditions, such as severe weather or un-

usually high airport traffic. Instead, Isolation Forest is used as a diagnostic method, and an additional outlier flag is added as a feature for modeling. A cardinality check is also performed because the dataset is large and contains high-cardinality categorical variables, especially airport-related features. The relationship between features and the target variable is also analyzed using correlation and mutual information.

- 5) **Preprocessing and Modeling.** This section defines the feature set, target variable, train-test split, preprocessing logic, and model pipelines. Pipelines are used because they make it easier to combine preprocessing and modeling in a consistent way. Different models require different preprocessing strategies: for example, KNN needs scaling because it is distance-based, while tree-based models can work directly with unscaled numerical features. Categorical variables are handled using a combination of one-hot encoding and target encoding, depending on their cardinality and model requirements.
- 6) **Evaluation of Models.** This section compares model performance using classification metrics such as accuracy, precision, recall, and F1-score. Since the target classes are imbalanced, accuracy alone is not sufficient, so recall, precision, F1-score, and ROC-AUC are especially important for evaluating how well the models identify delayed flights. ROC curves are used to compare the models' ability to separate delayed and on-time flights across different classification thresholds.
- 7) **Model Interpretation.** For tree-based models, feature importance and SHAP values are used to interpret model behavior. These methods help identify which variables contribute most to the predictions and make the results more explainable. In this project, features such as departure hour, precipitation, and departing airport appear to be among the most influential predictors of flight delay.

2. Proposed Methods

2.1. Data Understanding

This section focuses on exploring the structure of the dataset and identifying initial patterns related to departure delays. For better readability and interpretation, several variables are transformed into more understandable formats: months are represented in a clear 1–12 format, days of the week are converted into readable day labels, the departure hour is extracted from the departure time, and temperature values are converted from Fahrenheit to Celsius.

The first step is to analyze the target distribution. The dataset contains about 81% on-time flights and 19% delayed flights, which means that the classes are imbalanced. This is an important finding because it affects model training and evaluation: accuracy alone may be misleading, so met-

rics such as precision, recall, F1-score, and ROC-AUC are needed later.

Next, bar charts are built for categorical variables together with delay rate overlays. This makes it possible to compare not only the number of flights in each category, but also the share of delayed flights. At this stage, a clear relationship between departure time and delay rate is observed: flights departing later in the day tend to have a higher delay rate. This leads to the hypothesis that departure hour should be an important predictive feature in the machine learning models.

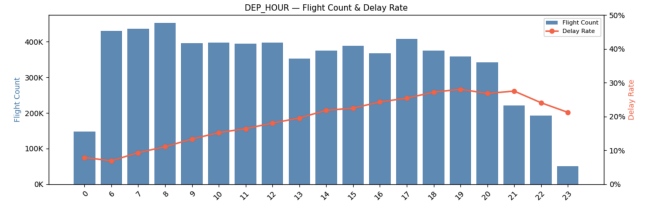


Figure 1. Categorical histogram: Departure Hour

Boxplots are then used to compare the distributions of numerical features for on-time and delayed flights. The goal is to understand whether delayed flights differ from non-delayed flights in terms of weather, traffic, aircraft, or route-related variables. The plots show that most numerical features have similar distributions for both classes. The most visible difference appears in `DEP_HOUR`, where delayed flights are more often associated with later departure times. Smaller differences are also observed in weather-related variables such as `SEGMENT_NUMBER`, and `NUMBER_OF_SEATS`.

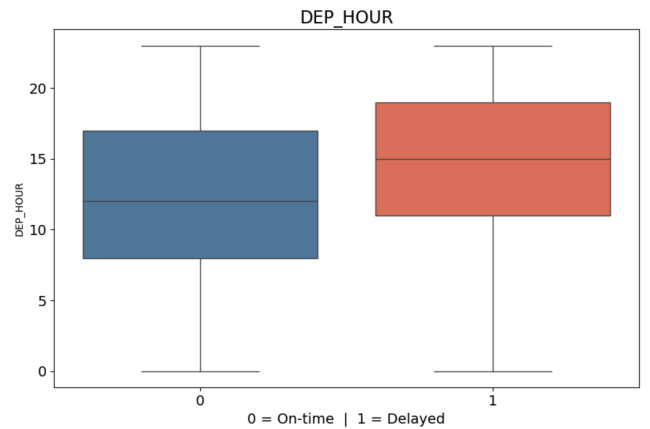


Figure 2. Feature distribution: Departure Hour

For `SNOW`, both delayed and on-time flights are mostly concentrated around zero, meaning that most flights took place without snowfall. However, delayed flights show a slightly heavier upper tail, suggesting that snowfall events are rare but may be more connected with delays when they occur.

2.2. Data Quality Analysis

The Data Quality Analysis section checks whether the dataset is reliable enough for modeling and whether any cleaning or transformations are required before training machine learning models.

First, missing values were examined. The dataset did not contain any NaN values. However, there were records where `PREVIOUS_AIRPORT` had the string value `NONE`. These records were analyzed separately to check whether an empty previous airport always corresponds to the first flight in the aircraft sequence. This check was important because `NONE` in this column can represent meaningful operational information rather than a missing value.

Next, several data sanity checks were performed. Numerical columns were checked for impossible negative values, except for temperature-related variables, where values below zero can be valid depending on the measurement scale and weather conditions. Suspicious zero values were also reviewed, especially in weather columns.

The `SNOW` variable was analyzed in more detail to verify whether snowfall appeared in reasonable months and locations. Snow was not observed in summer months, which supports the reliability of this feature. Some snowfall records appeared in April and May. May snowfall records were all linked to Denver International (Stapleton), where late spring snow is common. April snow events were distributed across several airports at mid-to-high latitudes and were treated as plausible given the geographic diversity of US airports.

Temperature values were also reviewed by month. The distribution followed the expected seasonal pattern, with the highest maximum temperatures appearing during summer months. This supported the assumption that the weather data was generally consistent and meaningful.

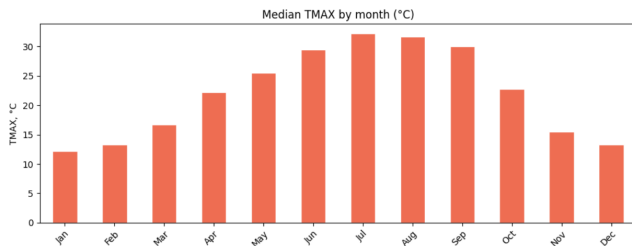


Figure 3. Median maximum temperature by month

Overall, no clear data anomalies were found in the dataset. In this context, an anomaly would mean an impossible or highly unrealistic value, such as an extreme temperature of 100 degrees Celsius, negative passenger or flight counts, or snowfall in an impossible seasonal and geographic context. The checks showed that unusual values mostly represented plausible real-world conditions, so the dataset was considered trustworthy for further preparation and modeling.

However, weather variables represent the maximal value for a day, which introduces potential **data leakage**: at the time of departure, the daily maximum may not yet have

been observed. As no date or flight ID columns are available in the dataset, it is not possible to substitute lagged weather observations. Therefore, these features should be treated with caution: they are informative, but in a real-time production setting they would need to be replaced with weather forecasts or the last available value.

2.3. Data Preparation

The section focuses on preparing the dataset for modeling and deciding which transformations are appropriate for different algorithms.

First, outliers were inspected. In a flight delay dataset, outliers are not unusual: extreme values may reflect real operational situations, such as severe weather, very high airport traffic, unusual route characteristics, or exceptional aircraft activity. Because tree-based models are generally robust to outliers, these observations were not removed from the main dataset. For KNN, which is distance-based and therefore more sensitive to scale and extreme values, feature scaling was planned and applied later in the modeling pipeline.

Isolation Forest was used as an additional diagnostic method for outlier detection. This method was used for attributes, where outliers are possible (month, day of week etc. were excluded from IF). The contamination threshold was set to 5%, meaning that the algorithm flagged approximately 5% of observations as potential outliers. The anomaly score distribution showed a relatively narrow left tail of more anomalous observations, while most records were concentrated in the normal range. This suggests that the majority of the dataset is consistent, while the detected outliers represent a small group of unusual cases rather than a broad data quality problem. Instead of deleting these records, an outlier flag was later added as an additional feature.

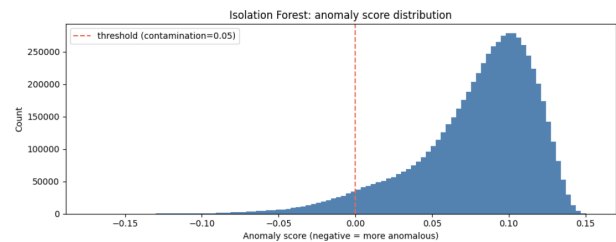


Figure 4. Anomaly score distribution

Next, feature cardinality was checked. This step was important because the dataset is large and contains categorical variables with many unique values, especially airport-related columns. High-cardinality features can become a bottleneck for one-hot encoding, creating too many columns and increasing memory usage and processing time. Therefore, different encoding strategies were considered for low- and high-cardinality categorical variables.

The relationship between features and the target variable was then examined. Pearson correlation showed that most numerical features had no strong linear relationship with

DEP_DEL15. Because the target patterns may be non-linear, mutual information was also calculated. The mutual information scores were low in absolute terms, but they were still useful for ranking features and identifying which variables carried relatively more predictive signal.

DEP_DEL15	1.000
DEP_HOUR	0.164
SEGMENT_NUMBER	0.116
PRCP	0.080
AWND	0.051
SNOW	0.050
AIRPORT_FLIGHTS_MONTH	0.026
LONGITUDE	0.026
SNWD	0.026
AVG_MONTHLY_PASS_AIRPORT	0.024
DISTANCE_GROUP	0.016
AIRLINE_AIRPORT_FLIGHTS_MONTH	0.014
NUMBER_OF_SEATS	0.012
CONCURRENT_FLIGHTS	0.009
PLANE_AGE	0.006
AIRLINE_FLIGHTS_MONTH	0.003
AVG_MONTHLY_PASS_AIRLINE	0.001
DAY_OF_WEEK	-0.000
LATITUDE	-0.000
FLT_ATTENDANTS_PER_PASS	-0.002
TMAX_C	-0.009
GROUND_SERV_PER_PASS	-0.017
MONTH	-0.019
Name: DEP_DEL15, dtype: float64	

Figure 5. Pearson correlation

Finally, additional engineered features, such as an indicator for the first flight, were created. Duplicate or highly correlated features were removed to reduce redundancy and noise. For example, variables such as AIRPORT_FLIGHTS_MONTH and AIRLINE_AIRPORT_FLIGHTS_MONTH were strongly related, so keeping both could add unnecessary noise without providing much new information. Redundant date and time features were also removed, such as keeping DEP_HOUR instead of the original departure time block. This helped simplify the feature set before modeling.

2.4. Model-Specific Preprocessing and Modeling

The goal of this section is to build models that are robust enough to work with a large real-world dataset and can produce reasonable predictive performance within an acceptable computation time.

To keep the modeling process consistent and easier to manage, preprocessing and model fitting were organized using pipelines. This allowed each model to have its own preprocessing strategy while keeping the training and prediction workflow structured. In majority of pipelines, robust scaling was used to handle numeric scaling, one-hot encoding for low-cardinality features, and target en-

coding for high-cardinality ones. To address class imbalance (~ 19% delayed flights), models were trained with `class_weight="balanced"`.

For computationally expensive models, such as KNN and RIPPER, a sample of 100K training observations and 20K test observations was used. Faster and more scalable models, especially tree-based models, were later trained and evaluated on the full dataset. A fixed `random_state=42` was used throughout the project to make the results reproducible.

The first model tested was **K-Nearest Neighbors (KNN)**. Since KNN is a distance-based algorithm, numerical features were scaled using robust scaling, and categorical variables were encoded before training. KNN was useful as a baseline, but it showed low recall and signs of overfitting, meaning it did not generalize well to unseen data.

A second KNN version was tested with **PCA**. PCA was added to reduce dimensionality and speed up distance-based learning. However, PCA did not improve the results. Since PCA preserves overall variance rather than target-specific information, it likely removed some predictive signal.

The **RIPPER** rule-based classifier was tested to see whether simple interpretable rules could identify delayed flights. The learned rules were too specific and did not provide strong predictive performance.

Gaussian Naive Bayes was used as a probabilistic baseline model. The preprocessing strategy was the same as for KNN algorithm. GaussianNB assumptions were too simple for this dataset, so the overall F1-score remained limited.

The **Decision Tree** model was then tested because it can capture non-linear relationships and does not require feature scaling. It was first evaluated on a sample and then on the full dataset. Feature importance scores and SHAP values were computed to interpret the model, identifying which features drove the predictions most strongly.

The **Random Forest** model was trained on the full dataset as an ensemble of decision trees, extending the single-tree approach with bagging to reduce overfitting. Feature importance scores were extracted to compare which variables the ensemble relied on most relative to the Decision Tree.

Finally, **LightGBM** was tested as a gradient boosting model suited for large tabular data. Two versions were built: the first trained on the full dataset with the base feature set and an extended version with additional feature engineering. It included departure hour and plane age bins, route and carrier-airport interaction features, and weather condition features. Weather variables were treated as real-time information, assuming that their values would be available at the time of prediction. Hyperparameters were tuned with RandomizedSearch on a stratified sample.

2.5. Results

Flight delay prediction is a challenging binary classification task. During exploratory analysis, most numerical features showed strong overlap between

on-time and delayed flights. Variables such as `CONCURRENT_FLIGHTS`, `NUMBER_OF_SEATS`, `PLANE_AGE`, and `DISTANCE_GROUP` did not clearly separate the two classes on their own. This suggests that individual features have limited predictive power and that the model needs to capture combinations of temporal, weather, airport, and operational factors.

The analysis discovered that flights later in an aircraft's daily sequence were more likely to be delayed. This supports the idea of **delay propagation**: if an aircraft experiences delays earlier in the day, later segments may also be affected.

Delay rates vary across both **airports and carriers**, with the variation being more pronounced across carriers than airports. However, this operational context is not directly reflected in the top feature importance results: departure hour and precipitation were among the most influential predictors. Airport-related traffic variables appeared further down the ranking, while carrier identity had a smaller direct contribution. This suggests that **when** a flight departs and **weather conditions** at departure are stronger signals than which airline or airport is involved.

Weather variables were analyzed separately because their interpretation depends on data availability. In the dataset, weather features represent daily observed values, such as maximum daily temperature, precipitation, snowfall, snow depth, and average wind speed. Therefore, they were inspected during exploratory analysis, but not used in the main models to avoid potential leakage. Weather features were included only in the final LightGBM experiment, under the assumption that they would be available at prediction time as real-time or latest observed weather information. In this setup, weather variables appeared among the most important predictors, especially precipitation-related features.

Several **baseline models** were tested and compared using precision, recall, F1-score, and ROC-AUC. **Recall** was treated as the primary metric, as the practical goal is to identify as many delayed flights as possible. For tree-based and gradient boosting models, feature importance scores and SHAP values were used to interpret which variables contributed most to the predicted delay probability.

KNN showed signs of overfitting, with substantially better performance on the training sample than on the test sample. KNN with PCA produced virtually identical results to plain KNN (ROC-AUC 0.595 vs. 0.593), confirming that PCA did not provide any meaningful improvement.

The rule-based **RIPPER** model showed poor performance, with low evaluation metrics. Accuracy was high, but only due to imbalanced classes. ROC-AUC value was 0.5, meaning it operated like a random classifier.

Gaussian Naive Bayes achieved lower accuracy than KNN (0.718 vs. 0.786) but substantially better recall for the delayed class (0.299 vs. 0.119 on the sample), making it more useful for the primary task of detecting delays. On the full dataset, recall improved slightly further to 0.330 with ROC-AUC of 0.615. Despite this, overall F1-score remained low (0.304), reflecting poor precision and confirming that GNB served as a simple probabilistic baseline rather than a competitive model.

Tree-based models were the most promising for this dataset. The **Decision Tree** achieved recall of 0.653 and ROC-AUC of 0.668 on the full dataset, a substantial improvement over KNN and GNB. **Random Forest** improved these results further: recall 0.695 and ROC-AUC of 0.673, confirming that ensemble methods reduce overfitting compared to a single tree.

Baseline version of **LightGBM** reached recall of 0.650 and ROC-AUC of 0.695. The second version improved precision and ROC-AUC further, reaching 0.32 precision and ROC-AUC 0.73.

Feature importance and **SHAP analysis** showed that tree-based models mainly relied on a small group of predictors: `DEP_HOUR`, `MONTH`, `DEPARTING_AIRPORT`, `SEGMENT_NUMBER`, and `PREVIOUS_AIRPORT`. Early departures reduced the predicted probability of delay, while later departures increased it.

After adding weather-related attributes, they appeared among the top features, suggesting that weather can be an important explanatory factor for flight delays.

The table below summarizes the performance of all models evaluated in this project.

model	train_set	accuracy	precision	recall	f1	roc_auc
LightGBM - Feature Engineered + Hyperparameters	full (1.3M)	0.67	0.32	0.65	0.43	0.73
LightGBM	full (1.3M)	0.64	0.29	0.65	0.40	0.69
Random Forest	full (1.3M)	0.58	0.27	0.69	0.39	0.67
Decision Tree (full)	full (1.3M)	0.60	0.27	0.65	0.38	0.67
Decision Tree (sample)	sample (20k)	0.59	0.26	0.64	0.37	0.66
GaussianNB	full (1.3M)	0.71	0.28	0.33	0.30	0.61
GaussianNB	sample (20k)	0.72	0.27	0.30	0.29	0.61
KNN	sample (20k)	0.79	0.32	0.12	0.17	0.59
KNN + PCA	sample (20k)	0.79	0.32	0.12	0.17	0.59
RIPPER	sample (20k)	0.81	0.33	0.00	0.00	0.50

Figure 6. Models' evaluation results

The comparison shows that LightGBM performs best on this dataset, especially when trained on the full data and combined with additional feature engineering and hyperparameter tuning. The feature-engineered LightGBM achieved the highest F1-score and ROC-AUC, indicating the best overall balance between detecting delayed flights and avoiding excessive false alarms.

3. Conclusion

This project addressed flight delay prediction as a **binary classification** task, with the goal of predicting whether a flight would be delayed by 15 minutes or more. The initial idea of predicting the exact delay duration in minutes was not implemented because the selected dataset did not contain a numerical delay-duration target. Therefore, the final scope was adjusted to classification using `DEP_DEL15` as the target variable.

The analysis showed that flight delay prediction is a difficult problem. Most individual features did not clearly separate delayed and on-time flights, and the overall predictive performance remained moderate. This suggests that delays are influenced by a combination of factors rather than by one dominant variable. The most useful signals

were related to **departure time**, **precipitation**, **departing airport**, and the aircraft's **segment number** during the day.

To improve prediction accuracy, future work should include real-time or latest observed weather data. Weather features appeared among the important predictors in the final LightGBM model, so collecting prediction-time weather information would make predictions potentially more accurate.

For datasets of this size, tree-based models are recommended because they scale better than distance-based or rule-based approaches and can capture non-linear interactions in tabular data. The results also show that their performance can be further improved through targeted feature engineering and hyperparameter tuning, as demonstrated by the final LightGBM model.

Additionally, the current evaluation relied on random train-test splitting, which does not reflect the temporal nature of the prediction task. Time-based validation, where the model is trained on past flights and tested on future ones, would better simulate real deployment conditions. Tuning the classification threshold could also improve the balance between precision and recall depending on the operational use case.

A key limitation of this project is that the dataset mainly captures scheduled and historical operational attributes, while many real causes of delays, such as real-time air traffic conditions, crew availability, aircraft technical status, and previous flight disruptions, are not fully represented. Future work could include these real-time operational variables or use an alternative dataset containing exact delay duration in minutes for a regression task.

References

- [1] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [2] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [3] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

Alina Iakubova
May 6, 2026